

Hybrid Cryptographic Monitoring System for Side-Channel Attack Detection on PYNQ SoCs



Nishant Chinnasami, Rasha Karakchi (Advisor)
Dept. of Computer Science and Engineering,
University of South Carolina, Columbia, SC, USA
nishantc@email.sc.edu, karakchi@cec.sc.edu

SEC

MOTIVATION & OBJECTIVE

- ❑ AES encryption, widely used in embedded systems, is increasingly vulnerable to power-based side-channel attacks, especially under real-time constraints.
- ❑ Traditional threshold-based detection methods often fail under high computational loads or complex scenarios. This work investigates machine learning (ML)-based anomaly detection pipelines that offer adaptive, scalable protection without disrupting encryption.
- ❑ The study compares ML methods with traditional techniques by evaluating their detection accuracy and adaptability across varying block sizes and malicious injection rates.

ML-BASED DETECTION ALGORITHM

Algorithm 2: AES Encryption with Anomaly Injection and ML-Based Detection

Input: N : number of plaintext blocks, p : malicious percentage, c : CPU cores
Output: Encrypted blocks, ML detection report

- Step 1: Generate Blocks**
- for $i \leftarrow 1$ to N do
- Generate random 16-byte block B_i
- Inject anomaly with probability $p/100$
- Step 2: Encrypt in Parallel**
- foreach $(B_i, i, anomaly)$ in parallel with c cores do
- if $anomaly$ then
- Randomly select anomaly type $\in \{\text{delay}, \text{fault}\}$
- if $delay$ then
- Sleep for random short time
- else if $fault$ then
- Flip first byte of B_i
- Pad/trim B_i to 16 bytes
- Encrypt B_i using AES-128-ECB
- Record encryption time t_i
- Step 3: Extract Features**
- Create dataset with timing t_i and original block bytes
- Step 4: Train ML Classifier**
- Split dataset into training and test sets
- Train Random Forest classifier on training data
- Predict labels on test set
- Step 5: Evaluate and Report**
- Calculate TP, FP, FN, accuracy
- Compute threshold $T = \text{mean}(t_i) + 3 \times \frac{\max(t_i) - \min(t_i)}{N}$
- Save results to Excel file

- * The ML model learns normal timing patterns from training data and classifies each block accordingly. It remains accurate under noise and varying workloads where threshold rules fail.

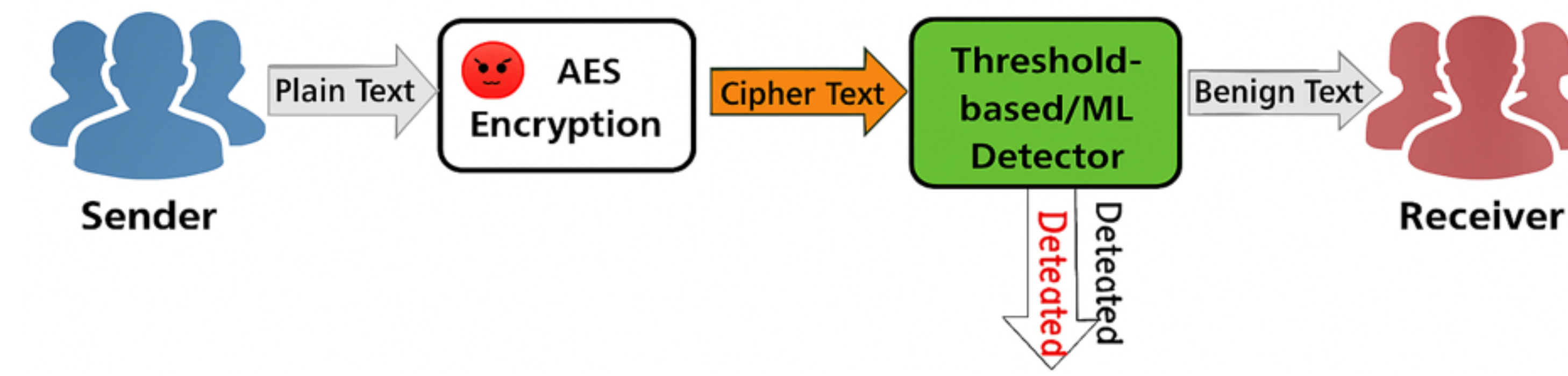
EMBEDDED PLATFORM

- Xilinx PYNQ-Z1 featuring a dual-core ARM Cortex-A9 CPU running lightweight Linux.
- All logic was implemented in native Python using onboard resources.

ACKNOWLEDGEMENT

This work was part of McNair Junior Fellowship and Magellan Scholar Grant by USC. The authors thank **Rye Stahle-Smith** for his assistance with hardware testing and experimental setup.

REAL-TIME DETECTION WORKFLOW



- *The system analyzes each AES encryption in real time by monitoring execution timing and ciphertext patterns. A threshold rule and lightweight ML classifier evaluate whether the behavior matches normal operation. Any abnormal timing or pattern deviation is flagged as malicious, enabling fast, low-overhead detection during live execution.

THRESHOLD-BASED DETECTION ALGORITHM

Algorithm 1: AES Encryption and Timing-Based Anomaly Detection

Input: Number of blocks n , CPU cores c , Anomaly ratio r
Output: Encrypted blocks, anomaly detection report

Generate n random plaintext blocks;
For each block: With probability r , inject anomaly (delay or fault);
Parallelize using c cores: For each block: if $anomaly == \text{delay}$ then
 Sleep for a short random time;
end
if $anomaly == \text{fault}$ then
 Flip first byte of block;
end
Pad/truncate block to 16 bytes;
Encrypt block with AES-128 in ECB mode;
Record encryption time;
Compute time threshold for anomaly detection;
For each result: if $\text{encryption_time} > \text{threshold}$ then
 Mark as malicious;
end
Compute detection metrics (accuracy, false positives, etc.);
Save results to Excel and display sample outputs;

- *The threshold method flags a block when its encryption time exceeds the normal range, providing fast, low-cost anomaly detection used as a baseline for ML comparison.

PERFORMANCE

Block Size	#CPUs	Latency (s)	Throughput	Memory (MB)
16384	1	0.0039	321.30	165.16
8192	1	0.0039	324.90	153.83
4096	1	0.0042	288.88	149.80
1024	1	0.0040	246.74	159.89
1024	2	0.0029	385.90	158.83
1024	4	0.0021	498.13	144.97

CPU EVALUATION

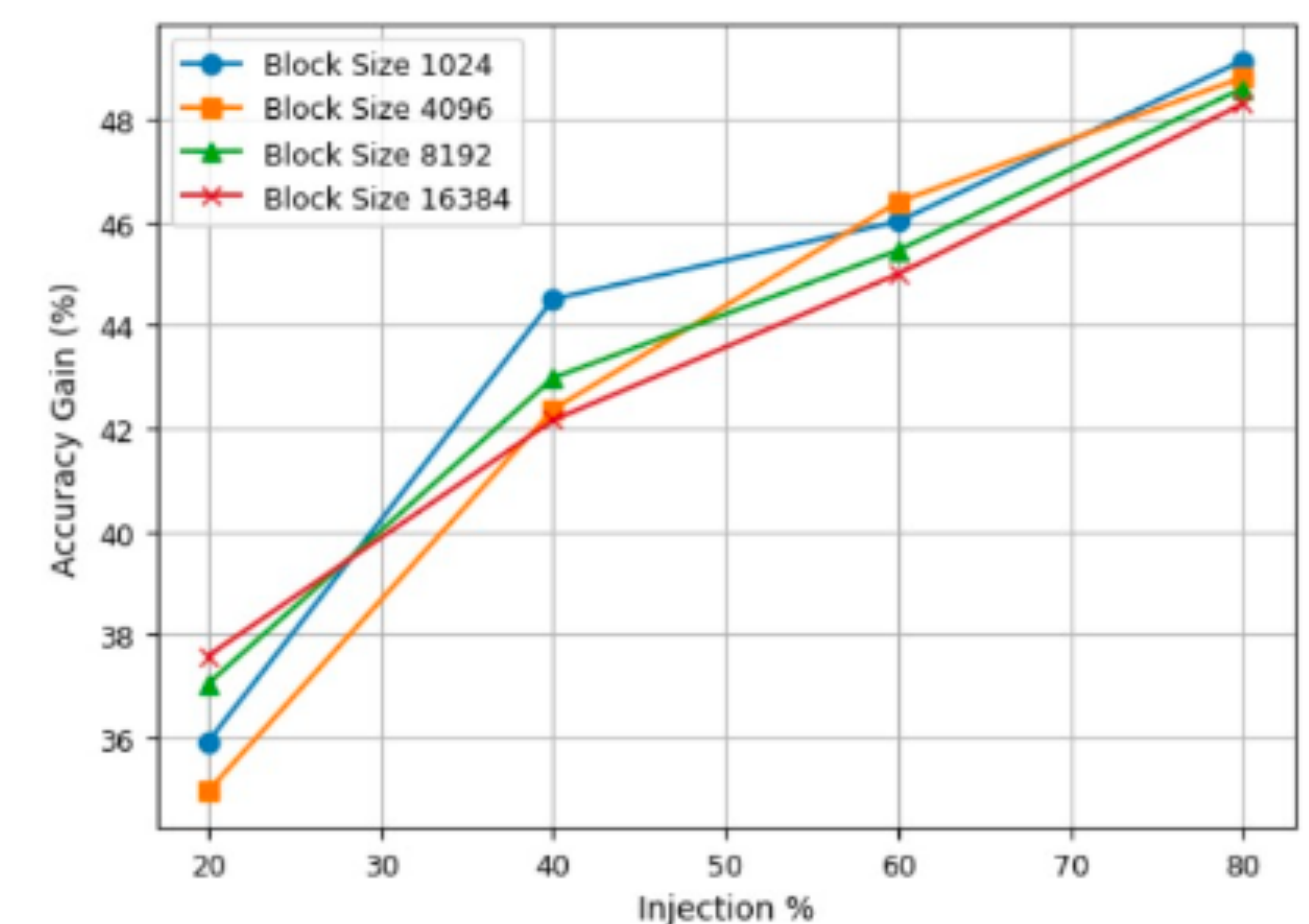
- We ran experiments on macOS 15.7 using Python 3.13.2 and AES-128 (ECB) from PyCryptodome.
- Execution time per block was used to simulate power features. Tests covered block sizes from 1,024 to 16,384, injection rates from 10% to 80%, and 1 to 8 CPU cores.
- Each run included AES encryption with timed malicious injection, followed by classification using either a threshold rule (based on execution time) or a DecisionTreeClassifier trained on the timing data.

DETECTION METRICS: ML VS THRESHOLD

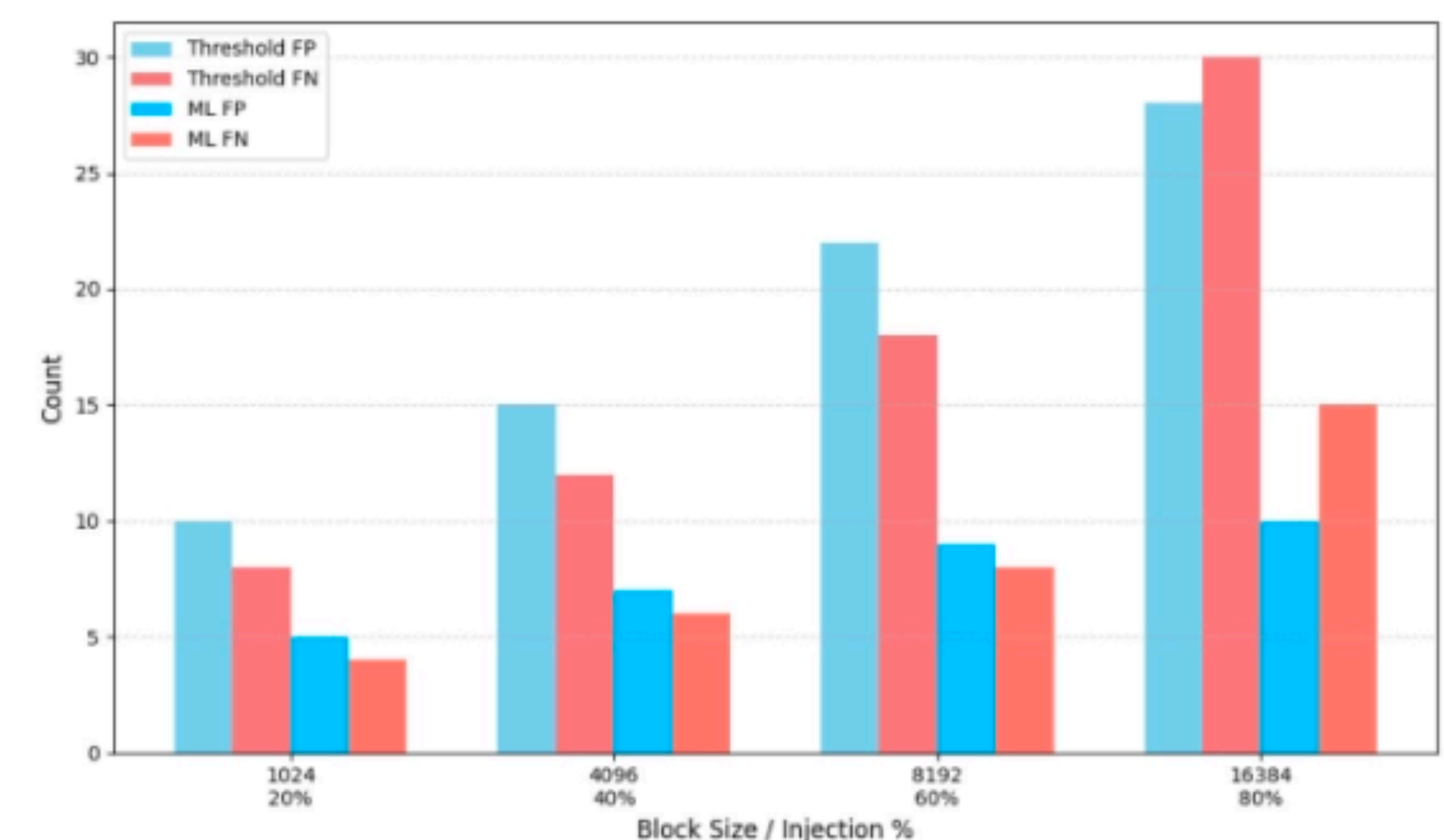
Block Size	Injection %	Recall	F1	Threshold Precision	ML Precision
1024	20	0.90	0.89	0.90	2.76
1024	40	0.83	0.83	0.83	2.80
1024	60	0.83	0.83	0.83	5.65
4096	20	0.89	0.87	0.89	6.21
4096	40	0.85	0.85	0.85	7.24
4096	60	0.83	0.82	0.82	7.75
8192	20	0.90	0.89	0.91	7.53
8192	40	0.84	0.84	0.87	7.75
16384	60	0.99	0.97	0.79	8.39
16384	20	0.99	0.94	0.91	8.39
16384	80	0.55	0.64	0.76	11.25

- * ML remains accurate under pressure while threshold breaks down as injection rises.

SCALABILITY



CLASSIFICATION



REFERENCE

- N. Chinnasami, R. Stahle-Smith, and R. Karakchi, "ML-Enhanced AES Anomaly Detection for Real-Time Embedded Security," *arXiv preprint* arXiv:2507.04197, 2025.
- Diffie-Hellman, R. S. A. "Timing Attacks on Implementations of." Advances in Cryptology—CRYPTO'96: 16th Annual International Cryptology Conference, Santa Barbara, California, USA, August 18–22, 1996, Proceedings. Vol. 1109. Springer, 1996.



Molinaroli College of Engineering and Computing
UNIVERSITY OF SOUTH CAROLINA